

Guide : Créer un serveur MCP en Python

Document mode opératoire pour concevoir, structurer et développer un serveur MCP (Model Context Protocol) en Python, basé sur le retour d'expérience du projet `mcp-teamleader`.

Table des matières

1. Qu'est-ce qu'un MCP ?
 2. [Architecture générale](#)
 3. [Initialiser le projet Python](#)
 4. [L'orchestrateur MCP \(`mcp\_server.py`\)](#)
 5. [Le business layer \(couche métier\)](#)
 6. [Gestion de l'authentification](#)
 7. [Bonnes pratiques](#)
 8. [Checklist de lancement](#)
-

1. Qu'est-ce qu'un MCP ?

Le **Model Context Protocol (MCP)** est un protocole ouvert créé par Anthropic qui permet à un LLM (comme Claude) d'appeler des **outils externes** de manière structurée.

Concrètement, un serveur MCP est un programme qui :

- **Déclare des outils** (tools) avec un nom, une description et un schéma JSON décrivant les paramètres attendus
- **Exécute ces outils** quand le LLM le demande, puis retourne le résultat sous forme de texte

Le LLM ne voit jamais le code source. Il voit uniquement :

- La **liste des outils** disponibles (nom + description + schéma des paramètres)
- Le **résultat** retourné après exécution

C'est pourquoi les descriptions et les schémas doivent être clairs et précis : c'est la seule documentation dont dispose le LLM pour utiliser correctement les outils.

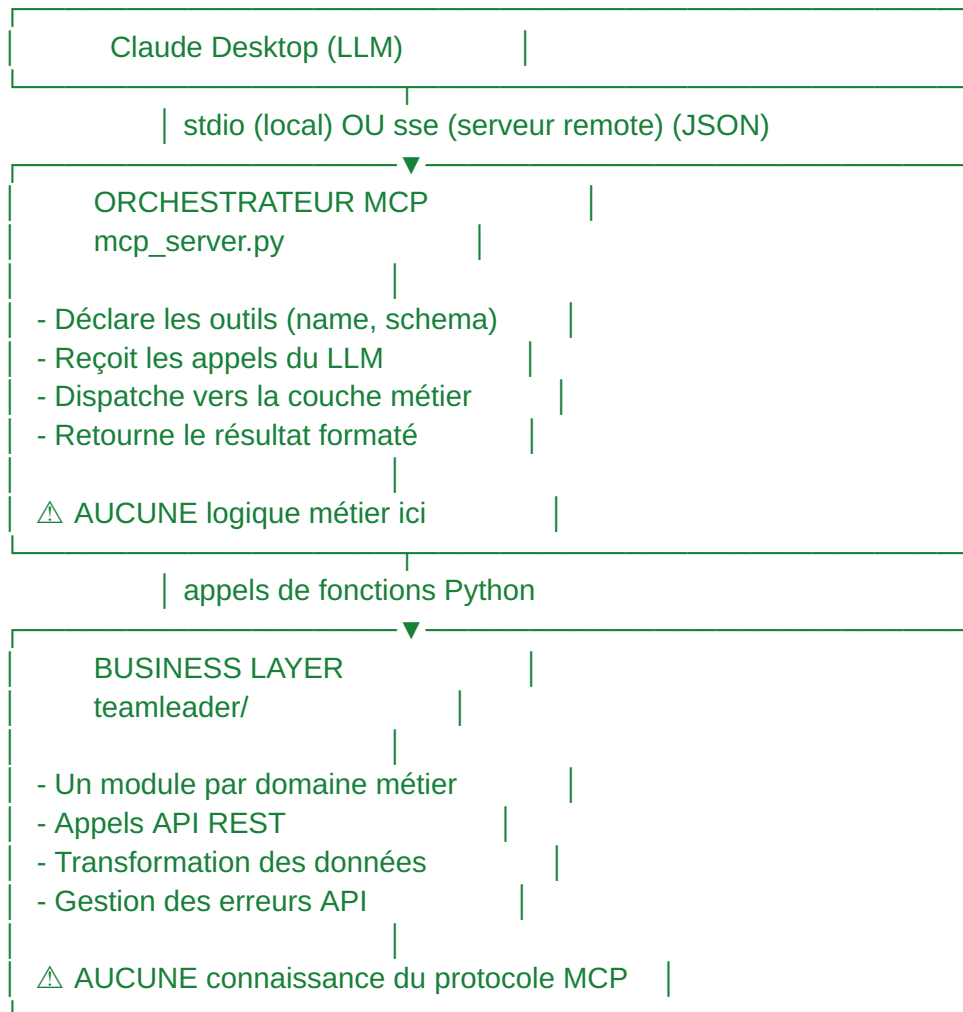
Transport stdio

Le serveur MCP communique avec Claude Desktop via **stdio** (entrée/sortie standard). Claude Desktop lance le processus Python et échange des messages JSON via stdin/stdout. Le développeur n'a pas à gérer ce protocole manuellement — le SDK `mcp` s'en charge.

2. Architecture générale

Principe fondamental : séparer transport et métier

Un serveur MCP bien conçu repose sur une séparation stricte en deux couches :



Pourquoi cette séparation ?

Sans séparation	Avec séparation
Toute la logique dans un seul fichier	Chaque domaine est isolé et testable
Modifier une API casse potentiellement le serveur MCP	Modifier un module métier n'impacte pas les autres
Impossible de réutiliser le code métier ailleurs	Le business layer peut être utilisé sans MCP

Difficile à debugger

On peut tester chaque fonction
indépendamment

Arborescence type

```
mon-mcp-server /
├── mcp_server.py      # Point d'entrée MCP du LLM en local - Point d'entrée pour la
                        recherche de l'outil à utiliser — enregistre et dispatch les outils [LOCAL + CLOUD]
├── cloud_run_server.py # Point d'entrée du LLM sur hébergement Cloud - Puis appel
                        mcp_server.py - Cloud Run — SSE, patches, OAuth cloud [CLOUD]
├── pyproject.toml     # Config du projet et dépendances
├── .env               # Variables d'environnement (local uniquement) (secrets)
├── .env.example       # Template .env (commité dans le repo)
├──
├── mon_api/          # Business layer (un package Python)
│   ├── __init__.py
│   ├── auth.py       # Authentification (tokens, refresh)
│   ├── contacts.py   # Domaine : contacts
│   ├── products.py   # Domaine : produits
│   ├── orders.py     # Domaine : commandes
│   └── ...
├── utils/            # Utilitaires transverses
│   ├── __init__.py
│   └── env.py         # Lecture/écriture .env
├── scripts/
│   └── ...           # Scripts annexes (réauth, migrations...)
└──
```

3. Initialiser le projet Python

3.1 Le fichier pyproject.toml

Le `pyproject.toml` est le fichier de configuration standard des projets Python modernes. Il remplace les anciens `setup.py` et `requirements.txt` en centralisant tout au même endroit.

```
[project]
name = "mon-mcp-server"
version = "1.0.0"
description = "Serveur MCP pour [API cible]"
requires-python = ">=3.10"
dependencies = [
    "mcp[cli]",
```

```

    "requests",
    "python-dotenv",
]

[build-system]
requires = ["setuptools"]
build-backend = "setuptools.backends._legacy:_Backend"

```

Explication des dépendances :

Package	Rôle
<code>mcp[cli]</code>	SDK MCP d'Anthropic — fournit <code>Server</code> , <code>Tool</code> , <code>TextContent</code> , le transport stdio
<code>requests</code>	Client HTTP pour appeler les API REST
<code>python-dotenv</code>	Charge les variables depuis le fichier <code>.env</code>

3.2 L'installation en mode éditable

```

# Windows
python -m venv venv
venv\Scripts\pip install -e .

```

```

# macOS / Linux
python3 -m venv venv
venv/bin/pip install -e .

```

La commande `pip install -e .` (mode éditable) fait deux choses essentielles :

1. **Installe les dépendances** listées dans `pyproject.toml`
2. **Rend les packages du projet importables** — les dossiers avec `__init__.py` deviennent des modules Python utilisables partout dans le projet

Sans cette étape, les imports comme `from teamleader.contacts import list_contacts` échoueraient, car Python ne sait pas où trouver le package `teamleader`.

3.3 Les fichiers `__init__.py`

Chaque dossier qui doit être importable en tant que package Python doit contenir un fichier `__init__.py` (qui peut être vide) :

```

teamleader/
├── __init__.py    # Rend le dossier importable comme package

```

```
|— contacts.py
|— companies.py
|— ...
```

```
utils/
|— __init__.py
|— env.py
```

Sans ces fichiers, `from teamleader.contacts import list_contacts` provoquerait une erreur `ModuleNotFoundError`.

3.4 Le fichier .env

Les secrets (clés API, tokens) ne doivent jamais être commités dans le dépôt. On utilise un fichier `.env`, utile quand on est sûr de l'utilisation du MCP en local :

```
# .env (ignoré par git via .gitignore)
API_CLIENT_ID=xxx
API_CLIENT_SECRET=xxx
API_ACCESS_TOKEN=xxx
API_REFRESH_TOKEN=xxx
```

Et un fichier `.env.example` commité comme template :

```
# .env.example (commité)
API_CLIENT_ID=
API_CLIENT_SECRET=
```

Chargement en Python avec `python-dotenv` :

```
from dotenv import load_dotenv
import os

load_dotenv()
client_id = os.getenv("API_CLIENT_ID")
```

4. L'orchestrateur MCP (mcp_server.py)

L'orchestrateur est le **seul fichier qui connaît le protocole MCP**. Il a trois responsabilités et rien d'autre :

1. Déclarer les outils disponibles (`list_tools`)
2. Router les appels vers les bonnes fonctions métier (`call_tool`)
3. Formater les réponses pour le LLM

4.1 Structure de base

```
import asyncio
import json

from mcp.server import Server
from mcp.types import Tool, TextContent

# Imports depuis le business layer
from mon_api.contacts import list_contacts
from mon_api.products import list_products

server = Server("mon-serveur")

# — Déclaration des outils —————

@server.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="list_contacts",
            description="...",
            inputSchema={...}
        ),
        # ...
    ]

# — Dispatch des appels —————

@server.call_tool()
async def call_tool(name: str, arguments: dict) ->
list[TextContent]:
    try:
        if name == "list_contacts":
            result = list_contacts(term=arguments.get("term"))
            return _ok(result)

        # ... autres tools ...

    return _err(f"Unknown tool: {name}")
```

```

        except Exception as e:
            return _err(str(e))

# ——— Helpers de formatage ———

def _ok(data: dict) -> list[TextContent]:
    return [TextContent(
        type="text",
        text=json.dumps(data, ensure_ascii=False, indent=2)
    )]

def _err(message: str) -> list[TextContent]:
    return [TextContent(type="text", text=f"Error: {message}")]

# ——— Point d'entrée ———

async def main():
    from mcp.server.stdio import stdio_server
    async with stdio_server() as (read_stream, write_stream):
        await server.run(
            read_stream,
            write_stream,
            server.create_initialization_options()
        )

if __name__ == "__main__":
    asyncio.run(main())

```

4.2 Déclarer un outil : l'inputSchema

Chaque outil est déclaré avec un **JSON Schema** qui décrit ses paramètres. C'est ce schéma que le LLM utilise pour construire ses appels.

```

Tool(
    name="list_contacts",
    description="List contacts, optionally filtered by name, email,
or phone.",
    inputSchema={
        "type": "object",
        "properties": {

```

```

        "term": {
            "type": "string",
            "description": "Search term: name, email, or phone
number."
        },
        "page": {
            "type": "integer",
            "description": "Page number (default: 1).",
        },
        "size": {
            "type": "integer",
            "description": "Results per page (default: 20).",
        },
    },
    "required": [] # Aucun paramètre obligatoire
}
)

```

Règles importantes :

Aspect	Recommandation
<code>name</code>	Nom technique, <code>snake_case</code> , unique parmi tous les outils
<code>description</code>	Claire et concise — c'est ce que le LLM lit pour décider d'utiliser l'outil
<code>properties</code>	Chaque paramètre avec son <code>type</code> et une <code>description</code> précise
<code>required</code>	Liste des paramètres obligatoires — si vide, tous sont optionnels
Nommage des paramètres	Utiliser des noms qui reflètent la réalité de l'API (ex: <code>term</code> plutôt que <code>name</code> si le filtre accepte nom/email/téléphone)

4.3 Le dispatch : router les appels

Le `call_tool` reçoit le nom de l'outil et un dictionnaire d'arguments, et route vers la bonne fonction :

```

@server.call_tool()
async def call_tool(name: str, arguments: dict) ->
list[TextContent]:
    try:
        if name == "list_contacts":
            result = list_contacts(

```



```

        term=arguments.get("term"),          # optionnel - .get()
        page=arguments.get("page", 1),      # optionnel
    avec défaut
        size=arguments.get("size", 20),
    )
    return _ok(result)

    elif name == "create_deal":
        result = create_deal(
            title=arguments["title"],        # obligatoire - []
            customer_type=arguments["customer_type"],
            customer_id=arguments["customer_id"],
        )
        return _ok(result)

    return _err(f"Unknown tool: {name}")

except Exception as e:
    return _err(str(e))

```

Convention :

- Paramètre **obligatoire** (`required`) → `arguments["param"]` — lève une erreur si absent
- Paramètre **optionnel** → `arguments.get("param")` ou `arguments.get("param", default)`

4.4 Ce que l'orchestrateur ne doit PAS faire

- Appeler directement des API HTTP
- Contenir de la logique de transformation de données
- Gérer l'authentification ou les tokens
- Importer `requests` ou autre client HTTP
- Contenir des boucles de traitement de données

Si on se retrouve à écrire plus de 3 lignes de logique dans un `elif` du dispatch, c'est le signe que cette logique appartient au business layer.

5. Le business layer (couche métier)

5.1 Un module par domaine

Chaque domaine fonctionnel de l'API a son propre fichier Python :

```
teamleader /
├── contacts.py    # Tout ce qui concerne les contacts
├── companies.py   # Tout ce qui concerne les entreprises
├── products.py    # Tout ce qui concerne les produits
├── deals.py       # Tout ce qui concerne les opportunités
├── quotations.py  # Tout ce qui concerne les devis
└── tax_rates.py   # Tout ce qui concerne les taux de taxe
```

Chaque module est **autonome** : il importe ce dont il a besoin et expose des fonctions publiques.

5.2 Structure type d'un module simple

```
"""
```

```
Module Contacts
```

```
Responsabilités :
```

```
- Lister et rechercher des contacts
```

```
"""
```

```
import requests
```

```
from mon_api.auth import get_valid_access_token
```

```
BASE_URL = "https://api.example.com"
```

```
def _headers() -> dict:
```

```
    """En-têtes HTTP avec le token d'accès valide."""
```

```
    return {"Authorization": f"Bearer {get_valid_access_token()}"}
```

```
def list_contacts(
    term: str = None,
```

```
    page: int = 1,
```

```
    size: int = 20
```

```
) -> dict:
```

```
    """
```

```
    Liste les contacts, avec filtre optionnel.
```

```
    Args:
```

```
        term: Terme de recherche (nom, email, téléphone)
```

page: Numéro de page (défaut: 1)
size: Résultats par page (défaut: 20)

Returns:

dict: Liste de contacts ou erreur
"""

```
payload = {"page": {"size": size, "number": page}}
```

```
if term:  
    payload["filter"] = {"term": term}
```

```
response = requests.post(  
    f"{BASE_URL}/contacts.list",  
    headers=_headers(),  
    json=payload  
)
```

```
if response.status_code != 200:  
    return {"error": f"API Error {response.status_code}:"  
{response.text}}"
```

```
data = response.json()
```

```
if not data.get("data"):  
    return {"contacts": [], "count": 0}
```

```
contacts = []  
for contact in data["data"]:  
    first = contact.get("first_name", "")  
    last = contact.get("last_name", "")  
    emails = contact.get("emails") or []  
    phones = contact.get("telephones") or []  
  
    contacts.append({  
        "id": contact.get("id"),  
        "full_name": f"{first} {last}".strip(),  
        "email": emails[0].get("email") if emails else None,  
        "phone": phones[0].get("number") if phones else None,  
    })
```

```
    return {"contacts": contacts, "count": len(contacts)}
```

5.3 Fonctions publiques vs privées (convention _)

En Python, le préfixe `_` indique qu'une fonction est **privée** — elle est destinée à un usage interne au module et n'est pas importée depuis l'extérieur.

```
# ✅ Fonction PUBLIQUE — importée dans mcp_server.py
def list_contacts(term=None, page=1, size=20) -> dict:
    ...

# ✅ Fonction PUBLIQUE — importée dans mcp_server.py
def get_quotation(quotation_id: str) -> dict:
    ...

# 🔒 Fonction PRIVÉE – utilisée uniquement dans ce fichier
def _headers() -> dict:
    ...

# 🔒 Fonction PRIVÉE – logique interne partagée
def _get_quotation_raw(quotation_id: str) -> dict:
    ...

# 🔒 Fonction PRIVÉE – transformation de données
def _normalize_grouped_lines(grouped_lines: list) -> list:
    ...
```

Règle : seules les fonctions publiques sont importées dans `mcp_server.py`.

Exemple d'import dans l'orchestrateur :

```
from teamleader.quotations import (
    list_quotations,          # publique
    get_quotation,           # publique
    create_quotation,        # publique
    update_quotation,        # publique
    add_product_to_quotation, # publique
    add_custom_line_to_quotation, # publique
    update_line_in_quotation, # publique
    delete_line_from_quotation, # publique
    add_contact_to_quotation, # publique
    add_company_to_quotation, # publique
)
```

```
# _get_quotation_raw, _normalize_grouped_lines, etc. → jamais importées
```

5.4 Structurer un module complexe

Quand un domaine a beaucoup de fonctionnalités (comme les devis dans `mcp-teamleader`), on organise le module en sections claires :

```
"""
Module Quotations

Responsabilités :
- Lister / consulter les devis
- Créer / mettre à jour les devis
- Ajouter / modifier / supprimer des lignes
- Lier un contact ou une entreprise au devis
"""

# -----
# List / Get
# -----

def _get_quotation_raw(quotation_id):    # privée, lecture brute
    ...

def _format_grouped_lines(raw):          # privée, formatage
    affichage
    ...

def list_quotations(deal_id=None, ...):  # publique
    ...

def get_quotation(quotation_id):        # publique
    ...

# -----
# Create / Update
# -----

def create_quotation(deal_id, ...):      # publique
    ...

def update_quotation(quotation_id, ...): # publique
```

```

...

# -----
# Add line item
# -----

def add_product_to_quotation(...):      # publique
    ...

def add_custom_line_to_quotation(...):   # publique
    ...

def _add_line_item(...):                 # privée, logique
    partagée
    ...

def _normalize_grouped_lines(lines):     # privée, conversion
    format
    ...

# -----
# Update / Delete line item
# -----

def update_line_in_quotation(...):       # publique
    ...

def delete_line_from_quotation(...):      # publique
    ...

# -----
# Link contact / company (via deal)
# -----

def add_contact_to_quotation(...):        # publique
    ...

def add_company_to_quotation(...):         # publique
    ...

```

5.5 Le pattern read-modify-write

Certaines API ne permettent pas de modifier un élément isolé — elles remplacent l'ensemble des données à chaque mise à jour. C'est le cas typique des lignes de devis dans Teamleader.

Le pattern read-modify-write consiste à :

1. LIRE → Récupérer l'état actuel complet
2. NORMALISER → Convertir le format de lecture en format d'écriture
3. MODIFIER → Ajouter / modifier / supprimer l'élément ciblé
4. ÉCRIRE → Envoyer l'ensemble modifié à l'API

Pourquoi une étape de normalisation ?

Beaucoup d'API retournent les données dans un format différent de celui qu'elles acceptent en écriture. Exemple concret :

```
# Ce que l'API RETOURNE (lecture) :
{
    "tax": {"id": "abc-123", "rate": 21.0},
    "unit_price": {"amount": 150.0, "currency": "EUR", "tax":
"excluding"}
}

# Ce que l'API ATTEND (écriture) :
{
    "tax_rate_id": "abc-123",
    "unit_price": {"amount": 150.0, "tax": "excluding"}
}
```

La fonction `_normalize_grouped_lines()` fait cette conversion automatiquement.

5.6 Retour d'erreur standardisé

Toutes les fonctions du business layer retournent un `dict`. En cas d'erreur, elles retournent `{"error": "message"}` au lieu de lever une exception :

```
def list_contacts(term=None) -> dict:
    response = requests.post(url, headers=_headers(), json=payload)

    if response.status_code != 200:
        return {"error": f"API Error {response.status_code}:
{response.text}"}
```

```
# ... traitement normal ...
return {"contacts": [...], "count": 5}
```

L'orchestrateur vérifie ensuite et transmet au LLM :

```
result = list_contacts(term="Dupont")
return _ok(result) # Sérialise en JSON, que ce soit un succès ou
une erreur
```

Le LLM reçoit soit les données, soit le message d'erreur, et peut adapter sa réponse à l'utilisateur.

5.7 Inter-dépendances entre modules

Un module métier peut appeler un autre module métier si nécessaire :

```
# quotations.py
from teamleader.deals import update_deal_customer
from teamleader.tax_rates import get_default_tax_rate_id

def add_contact_to_quotation(deal_id, contact_id):
    # Délègue au module deals
    return update_deal_customer(deal_id, "contact", contact_id)

def _add_line_item(..., tax_rate_id=None):
    if not tax_rate_id:
        # Récupère un taux par défaut depuis le module tax_rates
        tax_rate_id = get_default_tax_rate_id()
```

Règle : les dépendances vont toujours du module complexe vers le module simple, jamais l'inverse (pas de dépendance circulaire).

quotations.py → deals.py	✓ (devis utilise les deals)
quotations.py → tax_rates.py	✓ (devis utilise les taux)
deals.py → quotations.py	✗ (créerait une boucle)

6. Gestion de l'authentification

6.1 Module auth.py dédié

L'authentification est isolée dans un module dédié (`auth.py`). Ce module expose **une seule fonction publique** utilisée par tous les autres modules :

```
# auth.py
```



```
def get_valid_access_token() -> str:
    """
    Retourne un access token valide.
    Gère silencieusement le refresh si nécessaire.
    Déclenche la réauthentification complète si le refresh token
    expire.
    """
    ...
```

Chaque module métier l'utilise via la fonction `_headers()` :

```
# contacts.py
from mon_api.auth import get_valid_access_token

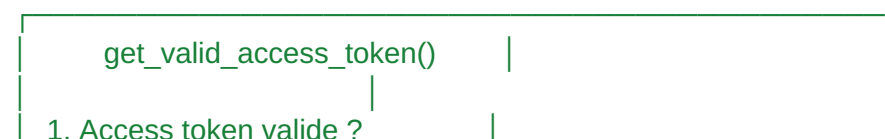
def _headers() -> dict:
    return {"Authorization": f"Bearer {get_valid_access_token()}"}

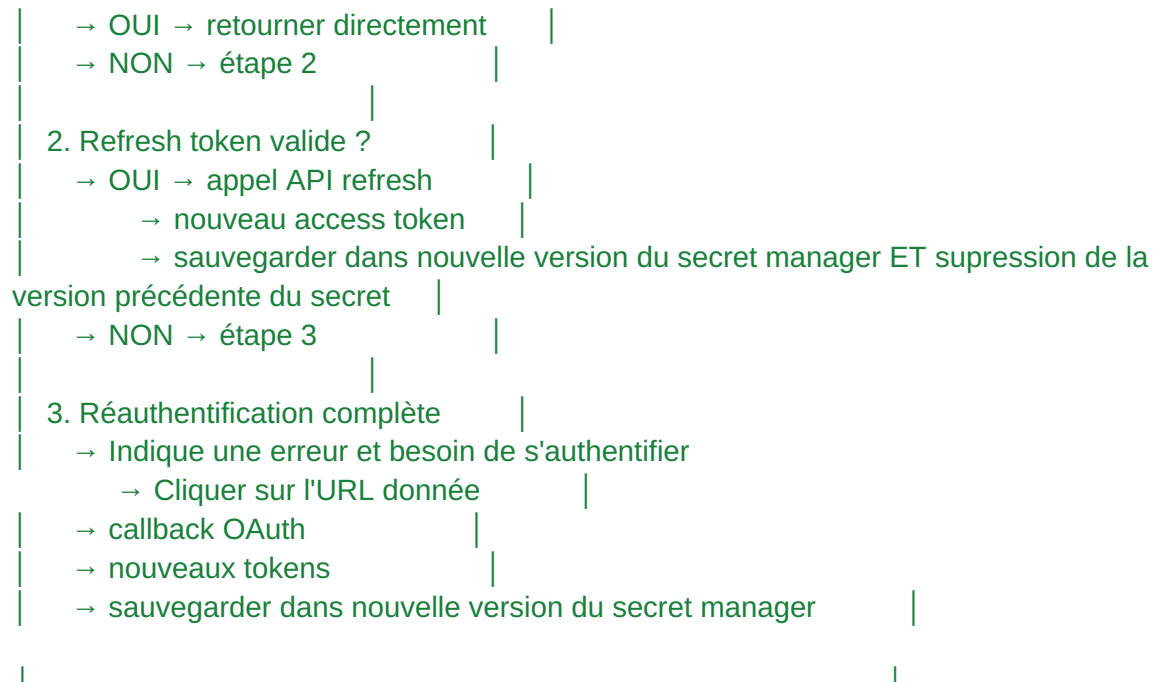
```

6.2.1 Cycle de vie des tokens en local (OAuth 2.0)



6.2.2 Cycle de vie des tokens en cloud (OAuth 2.0)





6.3 Règle clé : l'auth ne doit jamais fuiter

L'authentification ne doit jamais apparaître dans :

- L'orchestrateur (`mcp_server.py`)
- Les fonctions publiques du business layer

Elle est entièrement encapsulée dans `_headers()` qui appelle `get_valid_access_token()`. Si le token est expiré, le refresh se fait automatiquement, de manière transparente pour l'appelant.

7. Bonnes pratiques

Nommage des outils

Pratique	Exemple
Utiliser des verbes d'action	<code>list_contacts</code> , <code>create_deal</code> , <code>delete_line_from_quotation</code>
Être cohérent dans les préfixes	<code>list_*</code> pour lister, <code>get_*</code> pour un élément, <code>create_*</code> , <code>update_*</code> , <code>delete_*</code>

Nommer les paramètres
selon l'API

`term` si le filtre accepte nom/email/tel, pas `name`

Distinguer optionnel et
obligatoire

Un filtre de recherche est souvent optionnel, un ID est
obligatoire

Descriptions des outils

Les descriptions sont la **seule documentation** que le LLM a pour décider d'utiliser un outil.
Elles doivent être :

- **Précises** : "List contacts, optionally filtered by name, email, or phone" et pas juste "Get contacts"
- **Complètes** : mentionner ce que retourne l'outil ("Returns id, name, email, phone, country")
- **Honnêtes** : préciser les limitations ("Defaults to French 20% TVA if tax_rate_id is not provided")

Structuration des réponses

Retourner des dictionnaires avec des clés lisibles par le LLM :

 Bon : clés explicites, structure plate

```
return {  
    "contacts": [  
        {"id": "abc", "full_name": "Jean Dupont", "email":  
"jean@example.com"},  
    ],  
    "count": 1  
}
```

 Mauvais : données brutes de l'API non transformées

```
return response.json()
```

Le LLM travaille mieux avec des données propres, aplaties et nommées clairement.

Gestion de la pagination

Exposer `page` et `size` comme paramètres optionnels avec des défauts raisonnables :

```
def list_contacts(term=None, page=1, size=20) -> dict:  
    payload = {"page": {"size": size, "number": page}}  
    ...
```

Éviter les paramètres inutiles

Ne pas exposer un paramètre dans le schéma MCP s'il n'a pas de sens pour le LLM. Par exemple, `exchange_rate` est toujours 1 pour l'EUR — inutile de le montrer au LLM, on le gère en interne.

8. Hébergement MCP - Local - Cloud/VPS

Il existe 2 manières d'utiliser un MCP, soit en l'hébergeant en local ou en cloud. Ce choix va apporter des changements sur la structure et l'utilisation des variables d'environnement.

Pour l'utilisation d'un MCP en local :

- Téléchargement du projet
 - Installation des dépendances
 - etc.
- Modification de la configuration de Claude - Indiquer le chemin du MCP
- Communication/transport différent -> stdio - utilise le terminal
 - stdin = reçoit les messages
 - stdout = envoie les réponses
- Authentification - S'effectue automatiquement en silence (possibilité d'avoir accès au navigateur du pc)
- Lors de changement dans le code OU même les variables d'environnement, besoin de redémarrer Claude

Pour l'utilisation d'un MCP en Cloud :

- Indication dans Claude de l'URL du serveur contenant le MCP (Paramètre -> Connecteur -> Ajouter une intégration personnalisée)
- Communication/transport différent -> sse - HTTP
 - sse = reçoit les messages
 - messages = envoie les réponses
- Authentification - S'effectue via l'ouverture de la route /auth par l'utilisateur (pas possible d'accéder au navigateur)
- Lors de changement dans le code, besoin de repush une image ET redéployer
- Si changement de variable d'environnement, pas de besoin de mener de manipulation

9. Checklist de lancement

Structure du projet

- `pyproject.toml` avec les dépendances (`mcp[cli]`, `requests`, `python-dotenv`)
- `__init__.py` dans chaque dossier de package
- `.env` avec les secrets, `.env.example` comme template -> Pour le local
 - secret manager externe utilisé pour le cloud
- `.gitignore` excluant `.env`, `venv/`, `__pycache__/`

Orchestrateur (mcp_server.py)

- Import de toutes les fonctions publiques du business layer
- Chaque outil déclaré avec `name`, `description`, `inputSchema`
- Chaque `inputSchema` a des `description` sur chaque propriété
- Les `required` sont correctement renseignés
- Le dispatch (`call_tool`) route chaque outil vers la bonne fonction
- Les paramètres obligatoires utilisent `arguments["param"]`
- Les paramètres optionnels utilisent `arguments.get("param")`
- Aucune logique métier dans ce fichier

Business layer

- Un module par domaine
- Fonctions publiques = une par outil MCP (+ éventuelles fonctions partagées entre modules)
- Fonctions privées préfixées par `_`
- Chaque fonction retourne un `dict` (pas d'exception pour les erreurs API)
- Erreurs retournées sous forme `{"error": "message"}`
- Données transformées pour être lisibles par le LLM (pas de réponse API brute)

Configuration Claude Desktop/Browser - MCP hébergé en local

- `claude_desktop_config.json` pointe vers le `python.exe` du venv du projet
- Le chemin vers `mcp_server.py` est absolu
- Claude Desktop redémarré après modification de la config

Configuration Claude Desktop/Browser - MCP hébergé dans serveur Cloud ou VPS

- Besoin d'une version payante de Claude
- Paramètres -> Connecteurs -> Ajouter un connecteur personnalisé

Test

- Le serveur se lance sans erreur : `venv\Scripts\python mcp_server.py`
- L'authentification fonctionne au premier appel
- Chaque outil est visible dans Claude Desktop
- Un appel simple (ex: lister les contacts) retourne des données correctes